In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.

You know what *you* should own? A subscription to this newsletter:

FRED TALKS

18th August 2026

CONTENTS

Why compute might get 10x+ more expensive in coming years

DWARKESH PATEL · 29 JUL 2026 · SOURCE

If a human-level software engineer that could run on an H100 equivalent, at current market rates for software engineers, that H100 should rent for over $250k a year. That's 15x today's spot price.

I want to experiment with very quick blog post where I time-box writing for 2 hours. I won't be able to nail down a lot of important sub-questions, but the alternative is just not making any progress on a lot of different topics I'm curious about.

Today I want to talk about the compute situation of the labs over the coming years.

Anthropic revenue has 10xed year over year. Anthropic likely ends the year with ~$100–150B of revenue. For this trend to continue, Anthropic would have to make $1T in revenue by the end of next year. There's no deep reason why the trend needs to continue, and it very well might not - it's ultimately a question about AI capabilities. But suppose it does. What would have to be true about that world?

Lab compute 3x-es year over year. For a lab to 10x revenue while continuing to only 3x compute, some combination of the following 3 things has to happen: 1. Lab margins have to increase, 2. The price of compute has to increase, 3. Labs have to spend a greater fraction of their compute on inference.

My understanding is that basically all 3 of these things have been happening: 1. Anthropic went from 40% margins in 2025 to probably >80% this year for Fable inference[1] (though perhaps not on the marginal compute - see more below). 2. Spot prices for compute are up 40%+ from the

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?
- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.
- Do customers need to know? Who reported the bug? Who's blocked? Let them know.
- Does the world need to know? Announce that it's out.
- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.

"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?
- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?
- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?
- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here’s what I built, under these constraints, with these tradeoffs?”
- Test it manually. Yes, there’s automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

February trough, and that likely understates how much more the labs have to pay (again, more below). 3. Roughly a quarter of OpenAI’s 2024 compute spend went to inference according to Epoch , and it’s certainly closer to 50% if not higher now.

Labs would prefer *not* to do 3 (spend greater and greater shares of compute on inference). As has been said jokingly, the point of inference revenue is to convince investors to give you more money to buy more compute to train bigger models. If you’re spending most of your compute on inference, you’re kind of declaring that AI progress has stalled, because it’s not worth investing more in training, and your business is basically that of a cloud provider. The labs do not think this is true - they think they are serving models that will look extremely shitty within a year in order to build up the business case to continue training smarter models.

So that leaves 1. lab margins will increase, or 2. compute gets more expensive. It will be more of the former if the leading 1 to 2 labs are significantly ahead of the competition. In a market, your margins are set by how much better you are than the next best alternative. For effect 1 to dominate, margins would have to be in the mid-90s percent by the end of next year. That sounds quite crazy to me. But it does sound plausible to me that AI lab revenues will keep growing astonishingly fast.

So that leaves one more effect to explain how this world of crazy $1T revenue by end of next year might come to be: compute gets a lot more expensive. As I mentioned, this is already starting to happen. The price increase is even stronger when we look at the tranche of compute that the labs need - they obviously can’t rely on spot instances - they need security for their weights and customer info, and enough scale to get good utilization and flexibility. To look at how crazy the compute market is in that tranche, consider the price at which Google and Anthropic are renting compute from SpaceX. Google is reportedly paying $900 million a month for 110K GPUs that are a blend of GB200s and GB300s. That’s roughly 2x the spot price per hour for those GPUs. And the current spot price is itself 40% higher than it was in February.

I want to emphasize the key conclusion here: as AI models become smarter, they'll better monetize the same amount of compute. If a true human-level software engineer that could run on an H100 equivalent, at current market rates for software engineers, that H100 should rent for over $250k a year. That's 15x today's spot prices.

Of course you might expect that if we have 10 million extra software engineers, the marginal value of a software engineer would decrease and so that H100 wouldn't necessarily be able to produce 15x more revenue than it currently does. But I actually don't know if that's true. If we applied this argument to people instead of AI, then this would be the classic lump of labor fallacy . Economists generally believe that high-skilled immigration does not decrease wages in the long run because of how specialization and innovation increase the value of labor. Maybe this labor supply shock is so big and so fast that this general heuristic no longer applies. But if we believe what standard economics says about labor, then the marginal value of compute (and thus the marginal price of compute) should become astonishingly high.

What would happen in such a world?

- As the top models get better and better at monetizing compute, it becomes harder and harder to catch up. If by 2028 we've automated software engineering and the price of compute is 15x higher than it is right now, then it's going to be much more difficult for you, with no revenue, to compete for compute against the frontier labs.
- If you can train the best, most efficient model, then you'll be able to charge MUCH higher margins than you can today. This is the Alchian–Allen effect : when a fixed cost gets added to two goods of different quality, the premium good becomes relatively cheaper, so demand shifts toward it.
  At $20 per H100-hour, it's going to be extremely costly and stupid to use a weaker, less efficient model, because it's going to burn more tokens running your expensive compute to get the same result . So labs will be able to charge a very large premium if they can train a model that better economizes this scarce resource.
  If you're gonna have to pay so much for the underlying compute in the first place (directly or indirectly), you might as well pay extra to use the very best, most efficient model that runs on that same amount of compute.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company "veteran" capability remain with you even if a given "generalist" model is taken away?

Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek's sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it's imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: "What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it's not being transferred to someone else." The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization's data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm's right to align models to their enterprise accountability obligations.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what "good" looks like inside the organization. Also, retain ownership of your organization's memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company's knowledge.

- A lot of current popular applications of AI get priced out. The reason AI is relatively cheap right now, at least in comparison to human labor, is partly that it can't do a lot of things that top humans can do. At some point that will no longer be the case. And so using GPUs to make short-form video slop will just get priced out.
    - At the same time, this kind of prediction does pattern match onto the ways people have been wrong about scarcity in the past. I'm thinking of the Simon–Ehrlich wager, where Paul Ehrlich bet that the cost of a basket of commodities would increase rather than decrease in the decade leading up to 1990. This wager is famous in popular economics discussion, because it's supposed to illustrate how Ehrlich's Malthusian worldview was falsified — he underestimated the way in which market signals and human ingenuity would find ways to better economize scarce inputs. (Though other analysis shows that if this bet had been made in a different decade, Ehrlich would have won).
    - I'm guessing the Simon-Ehrlich basket of commodities is not the correct reference class for compute, because compute supply is much less elastic, and much less capable of absorbing large demand shocks, than the extraction of different metals is.
      That 3x in compute capacity per year comes from multiplying the following numbers together ~~,~~: 1.4x from Moore's Law, 1.2x from new fab construction (bottlenecked through at least 2030 by EUV tool supply), 1.8x from AI taking leading edge wafer allocation from other devices (which will start to hit a wall by end of 2027, when AI will go from 60% of N3 to 86%). Seems hard to shift any 3 of those inputs into compute scaling much more, and the final one will actually hit a will within a year or two as wafer allocation gets saturated.
- I want to clarify that at some point in the future compute becomes cheap again. At some point robots will be able to turn shores of silica sand and mines of copper into computers. At which point the price of compute should fall closer to the cost of raw inputs and tools. I'm just talking about this current regime where AI compute merely 3x-es year over year, which is not fast enough to offset the price effect of how much more useful AI is becoming year over year.
- By the way, the fact that Ant revenue has been 10x-ing year over year, while compute has only been 3x-ing, suggests that there are very strong economies of scale in the model business.
  Logically this makes sense - when you train a model, you pay this one time cost of learning all these different skills that you can then amortize across all your users.

(Unlike with human labor, where each instance has to be retrained from scratch).
I wish we didn't live in a world with such strong economies of scale of intelligence (because I'm worried about power concentration). But it seems we do.

1Blended inference margins are >70% , so I'm guessing Fable API margins are >80% - total vibe claim.

Upgrade to paid

View in app

How agents are transforming work | OpenAI

OPENAI · 28 JUN 2026 · SOURCE

June 25, 2026

Company

A new Economic Research paper measuring Codex's economic potential at the frontier.

Read the paper

Listen to article

[AUDIO OMITTED]

Share

Agentic AI changes the unit of knowledge work from single interactions to delegated, long-horizon tasks. Chatbot interactions are often short and self-contained. Agents can operate independently for minutes or hours while orchestrating tool calls, interacting with environments, and iterating towards solutions. As a result, agents are quickly becoming the most powerful AI tool for work.

Over the last year, we witnessed this transformation first-hand at OpenAI. For the first few months after Codex was released to the public, ChatGPT remained the default AI tool for work within OpenAI. Through August 2025, the average OpenAI worker spent less than 10% of their

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · SOURCE

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow famously described a paradox in the market for information. "Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost." In Arrow's "Information Paradox," the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow's paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from "exhaust," the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It's the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

tokens on Codex. Now, every department, including non-technical departments such as Legal and Recruiting, uses Codex as their primary AI tool for work. This pattern reflects what we believe will be the future of work given the expanded capabilities and accessibility of agentic tools.

Some content could not be imported from the original document. View content ↗

Some content could not be imported from the original document. View content ↗

Codex adoption grew in tandem with Codex's capabilities. As Codex leveraged stronger models and new product features, it became capable of taking on an expanding set of productive tasks. Across Individual users, Organizational users, and OpenAI workers, we document four trends over the past year:

- **People use Codex for longer-horizon work**. By May 2026, 80.6% of sampled individual users made at least one Codex request estimated to exceed 30 minutes of human work, 70.2% made one estimated to exceed one hour, and 25.6% made at least one Codex request estimated to exceed eight hours.

- **Codex became the primary AI tool for every department at OpenAI**. Engineering moved first, but Legal, Finance, and Recruiting crossed into Codex being their primary AI tool around April 2026. For the average OpenAI worker, Codex usage now accounts for more than 85% of output tokens. Since Codex users tend to use more tokens than non-users, its share of overall tokens is even higher: Codex accounts for 99.8% of weekly output tokens generated within OpenAI.

- **Non-developer adoption grew especially rapidly, outpacing developer adoption**. Since August 2025, non-developer users rose 137x for individual users, 189x for organizational users, and 12x within OpenAI.

- **Codex enabled OpenAI workers to do tasks outside their job description**. While technical usage is still most prevalent among engineers, non-technical users regularly use Codex to take on coding or technical execution, including automation, data transformation, tooling, debugging, and structured analysis.

Agents work longer hours on harder tasks

Nearly a quarter of all Codex requests are for tasks that would take a person more than one hour to complete[(1)]. As Codex's capacity for independent long-context work improved, users shifted from short interactions toward more difficult tasks with longer horizons.

The chart below estimates the share of individual users that crossed four human-time thresholds: tasks that would take a person more than 30 minutes, more than one hour, more than four hours, and more than eight hours(2). From December 2025 to May 2026, the share of users who made a request that was estimated to correspond to work that would take a person more than 30 minutes rose to 80.6%. The share making a request that would take a person more than one hour rose to 70.2%. The share requesting work that would take a person more than eight hours grew the fastest from a low base.

The growth in agentic usage can be seen in daily Codex runtime. Among daily active users at OpenAI, the heaviest users ask Codex to run many hours of agent work in a single day. By June 2026, users at the 99th percentile regularly generated more than 60 hours of Codex agent turns per day, distributed across multiple, parallel agents. As Codex became more powerful and parallelizable, users moved from only asking Codex for one answer at a time into increasingly orchestrating multiple agent tasks over the course of a day.

Adoption continues to move from engineers to the rest of OpenAI

Engineers at OpenAI began adopting Codex first, gradually. The average engineer at the company shifted the majority of their usage of OpenAI products to Codex by December 2025. Today, the average engineer generates 99% of their output tokens with Codex rather than ChatGPT. Legal, finance, and recruiting crossed over to majority use of Codex later, around April 2026, but their transitions were much faster. The average lawyer or recruiter at OpenAI now generates more than 85% of their output tokens on Codex.

Over the last six months, Codex usage has deepened and intensified at OpenAI. Among active internal users, change in combined output tokens rose sharply across departments. Research saw the biggest jump: by June 2026, median use was 56 times higher than in November 2025. Customer Support rose 32 times and Engineering rose 27 times, while Legal grew more gradually but still reached 13 times its November level.

These two patterns together illustrate how Codex has transformed how OpenAI uses AI to do productive work. Across the company, users are switching from chatbots to agents as their primary form of AI interaction and are deploying an exponentially growing amount of agentic labor.

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

Daybreak: Tools for securing every organization in the world

SecurityJun 22, 2026

Samsung Electronics brings ChatGPT and Codex to employees

CompanyJun 21, 2026

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Non-developers are the fastest-growing user groups

Across all user groups—OpenAI, organizational, and individual users—Codex use began with developers, the natural target audience for what began as a coding tool. However, as Codex expanded toward more general knowledge work, adoption among non-developers grew even more quickly. As shown in the user growth chart below, weekly non-developer users rose faster than developer users among individual, organizational, and OpenAI populations. By early June 2026, non-developer individual users multiplied 137 times since August 2025. Non-developer organizational users increased 189-fold, and non-developer OpenAI users increased 12-fold, likely because this group already started at a well above average starting point.

The shift does not mean every non-developer is using Codex in the same way as an engineer. Rather, it means that more non-developers are using Codex for some kind of agentic work.

Codex is expanding the horizon of potential productive work

Codex enables non-technical departments to accelerate workflows previously bottlenecked by technical expertise. The heat map below compares inferred occupations within OpenAI to the type of work represented in Codex outputs. Engineering and coding show up as the largest category for data science and research, whereas knowledge work is the largest category for finance and business operations, marketing, operations, and other departments.

That said, agentic tools can expand what an individual worker can do. For instance, over one-fourth of work done with Codex by workers in business functions was engineering or coding. Agents can lower the cost of moving across task boundaries and help workers do adjacent work that used to require more specialized technical support.

Occupation vs. work done with Codex

	Work category				
Consolidated inferred department	**Engineering / Coding**	**Data Analysis**	**Financial Analysis**	**Knowledge Work**	**Other**
Engineering	72%	4%	1%	18%	5%
Data Science / Research	51%	10%	0%	30%	9%
Finance / Biz Ops	31%	9%	16%	34%	10%
Product / Marketing / Ops	25%	3%	7%	51%	15%
Other	50%	7%	2%	38%	4%

Share of output tokens within department

What this means for agents' economic potential

Increased use of agentic tools by non-engineer employees expands the frontier of what these workers can do. That matters for businesses deciding how to redesign workflows, for employees learning which skills become more valuable, and for policymakers and researchers trying to understand how AI changes the labor market.

Our paper shows how frontier users adopt capable agentic tools at the frontier. Our results demonstrate what unfolds when people have broad, low-friction access to capable agentic tools: as the tools improve, people use them for longer, more complex, and more cross-functional work. As time goes on, this is likely to be what the future of work looks like.

- 2026
- Economic Research
- Codex

Author

OpenAI

Footnotes

1. Task horizon is estimated using an LLM-as-judge with access to Codex transcripts. [SVG OMITTED]

   [find in text]

2. These thresholds are model-estimated, so they should be treated as directional rather than exact. This is also based only on individual data and based on the queries of a random sample of 0.1% of users. [SVG OMITTED]

   [find in text]

Keep reading

View all

OpenAI and Broadcom unveil LLM-optimized inference chip

CompanyJun 24, 2026